

문자열 실습 과제

- 제출 방식: 10/17(토) 23:00까지 제출
 - 문제를 1.StringAPP / 2.MyString 폴더에 제출. 제출압축파일에 각 폴더 바로 포함.
- 유의사항
 - 1번은 각 기능별 클래스로 구현해도 되고, 클래스 없이 일반 함수로 구현해도 됨. 반드시 main() 포함하여 제출.
 - 알파벳 개수 카운트 및 출력, 대소문자 변경, 암호화는 반드시 각각의 함수로 구현할 것
 - 클래스를 사용하면 멤버함수, 아니면 일반함수로 구현하면 됨.
 - cstring(문자배열)과 <cstring> 사용해도 되고, <string>의 string 클래스는 사용 금지.
 - 2번은 <string.h>, <cstring>, <string> 등의 문자열 라이브러리 함수 사용 금지
 - strcpy, strcat 사용 금지. 입력에서 cin.get()으로 한글자씩 입력받을 것
 - 정적배열을 사용하므로 글자수가 15 초과인 경우 나머지 부분은 무시
 - '\0' 처리를 정확하게 할 것. 함수에서 const 선언 적절하게 할 것.
 - 정확성과 함께 가독성과 유지보수성이 가장 중요한 채점 기준임
 - 예를 들어 알파벳 개수에서 case 'A' : arr[0] ++ ; break ; case 'B' : arr[1] ++ ; break 를 쓰거나 간단암호화에서 if (gap == 3 && 'a' <= c && c <= 'w') 같은 방식으로 코딩하면 유지보수성이 매우 낮으므로 감점이 큼
 - 97보다 'a', 65보다 'A'가 가독성이 큰 점 참고할 것. const는 적절히 알아서 추가할 것.
 - 포인터 배운 뒤 동적배열의 resize()에 기반한 MyString 프로젝트 있으니, 설계 잘 하고 유지보수성 잘 고려

1. 문장 입력과 문자 처리

- 한 문장과 정수 하나를 입력받아 아래의 세가지 작업을 수행. 문자배열의 글자최대수는 127글자.
- 기능 1) 입력되는 문장에서 각 알파벳의 개수를 세어 출력.
 - 대소문자 구분 없이, a~z 순서로, 문장에 포함된 알파벳에 대해서만 출력
- 기능 2) 입력 문장에서 첫 글자는 대문자로, 나머지는 소문자로 변환
 - 입력의 앞 부분에 공백이 있을 수 있음. “첫 글자”가 대문자가 되어야 함.
- 기능 3) **알파벳과 숫자**를 다음 n번째 글자로 대체한 간단암호화 문장출력
 - Gap의 크기는 -25 ~ 25임. gap== -11 인 경우 ‘a’는 ‘p’로 변환, ‘o’은 ‘9’로 변환.
 - 아래는 gap 크기가 3이거나 -23인 경우의 암호화의 예 (숫자와 알파벳 외에는 변환하지 않음)

A	B	C	...	V	W	X	Y	Z
D	E	F	...	Y	Z	A	B	C

a	b	c	...	v	w	x	y	z
d	e	f	...	y	z	a	b	c

0	1	2	...	5	6	7	8	9	
3	4	5	...	8	9	0	1	2	gap = 3일 때
7	8	9	0	1	2	3	4	5	gap = -23일 때

This is A My 1-st Test PROGram.

3

[a:2] [e:1] [g:1] [h:1] [i:2] [m:2] ... [t:4] [y:1]

This is a my 1-st test program.

Wklv lv D Pb 4-vw Whvw SURJudp.

This is A My 1-st Test PROGram.

-23

[a:2] [e:1] [g:1] [h:1] [i:2] ...

This is a my 1-st test program.

Wklv lv D Pb 8-vw Whvw SURJudp.

2. class MyString의 구현

- 정적 배열 기반 최대 15글자 길이 문자열을 위한 class MyString 구현
 - main 은 제출하지 않음. MyString.cpp와 MyString.h만 제출. 적절한 main으로 충분히 테스트 해 볼 것
- 구현 대상1 (기본 수준)
 - MyString::MyString() : default 생성자
 - MyString::MyString(const MyString &srcStr) : srcStr로부터 복사하여 생성
 - MyString::operator = (const MyString &srcStr) : 대입연산자
 - int MyString::length() : 문자열 길이(전체 용량 아닌 문자열 자체 길이) 반환
 - bool MyString::empty() : 문자열이 비어있으면 true를, 아니면 false를 반환
 - char MyString::at(int pos), char MyString::operator[](int pos)
: pos 위치의 글자 return 또는 참조 (이번 과제에서는 MyString str ; str[0] = '\0' ; 같은 표현 미허용)
 - ostream& operator <<(ostream& os, const MyString &str) : 출력
 - istream& operator >>(istream& is, MyString &str)
: cin.get()으로 구현. whiteSpace까지 입력. 16 넘어서는 글자는 버퍼에 그대로 두기
 - istream& getline(istream& is, MyString &str, char delim='\n')
: delim 글자 들어올 때까지 까지 입력. get() 사용하여 구현. default delimiter 는 '\n'
: delim까지의 모든 글자를 읽되 16번째 글자부터는 버리기

2. class MyString의 구현 (cont.)

- 구현기능2 (필수 수준)
 - `MyString& MyString::operator = (const char str2nd[])` : str2nd를 this로 복사
 - 문자열 연결 연산자 오버로딩 또는 함수류 : this에 str2nd를 연결한 새 string 반환
 - `const MyString MyString::operator + (const char str2nd[])`
 - `const MyString MyString::operator + (const MyString& str2nd)`
: operator =와 operator + 모두 16글자를 넘어서면 15글자까지만 저장
 - 문자열 동등성을 비교하여 bool을 return하는 연산자 류: == 와 !=
 - `bool MyString::operator ==(const MyString &str)` `bool MyString::operator !=(const MyString &str)`
 - 부분문자열을 찾아 위치를 return하는 함수류. 부분 문자열이 없으면 -1 return
 - `int MyString::find(const char subStr[], int pos=0)`과
`int MyString::find(const MyString &subStr, int pos=0)`
: 문자열[pos] 위치 이후부터 subStr이 발생하는 시작 위치를 찾아 반환
: pos가 음수이면 0으로 취급, 문자열 길이 밖이면 -1 return
 - 부분문자열 추출 함수 `MyString MyString::substr(int pos, int len)`
 - 문자열 내에서 pos 위치로부터 len 개의 글자를 옮긴 새로운 MyString 객체 return
 - pos가 음수이면 0으로 취급, 문자열 범위 밖이면 빈문자열 return
 - len이 현재 문자열 길이에 맞지 않은 경우는 `string.substr()`의 동작과 동일하게 구현